STM32 Interrupt Programming

Ankit Srivastava

EE2028

ARMv7-M Architecture Ref Manual

RM0432 STM32L4+ Technical reference manual

PM0214 STM32 Cortex-M4 Programming Manual

Acknowledgement: Slides adapted from Dr Rajesh Panicker

OVERVIEW

- Interrupt Programming steps
  - PRIGROUP and IPR setting
  - ISR implementation
  - Device/peripheral interrupt configuration
  - Clear past interrupt events in NVIC
  - Enabling interrupt in NVIC
- Interrupt Handler invocation
- Example code

Interrupt Programming Steps

1. Set up the priority group setting
2. Set up the priority level of the interrupt
3. Implement the ISR
    i. Push all the registers modified by the handler
    ii. Identify the cause of interrupt
    iii. Do the bare minimum to deal with the cause
    iv. Clear the interrupt inside the interrupting device
    v. Pop all the registers pushed at the beginning
4. Configure the interrupting device and/or peripheral
5. Clear the pending status in NVIC
6. Enable the interrupt in NVIC

PRIGROUP AND IPR SETTING

- Set up the priority group setting
  - By default, priority group setting is zero (7 MSBs used for preempt priority)

- Set up the priority level of the interrupt
  - By default, all interrupts are at priority level 0 (highest)

ISR IMPLEMENTATION

i. PUSH all the registers modified by the handler, other than those which are automatically pushed by the hardware before the handler is invoked (R0–R3, R12, LR, PC, and xPSR – caller saved and special registers, as the 'caller' of ISR is h/w)
   - If ISR is implemented in C, this is taken care of implicitly

ISR IMPLEMENTATION

ii. Identify the exact cause / reason for the interrupt (continued in the next slide)

- If a peripheral generates interrupts due to multiple causes which go into NVIC as a single interrupt (i.e., the interrupt status bits are OR-ed together internally). This can typically be done by checking the interrupt status bit(s) of the peripherals
  - I2C_EV interrupt can be triggered due to several conditions such as RXNE, TXIS etc.
    - ❑ The exact condition can be determined by checking I2C_ISR
  - EXTI 5-9 are or-ed together and is recognized by NVIC as a single interrupt. Further, each pin can trigger an interrupt because of rising edge and/or falling edge
    - ❑ Identification of EXTI triggered and cause (rising/falling) can be done by reading EXTI_PR

ISR IMPLEMENTATION

ii. Identify the exact cause / reason for the interrupt

- If the interrupt is from an external device (e.g., a sensor) which is routed to NVIC through an EXTI, we might need to read appropriate register(s) within the device to find out the cause
  - This is done through the normal data channel connecting the external device to the processor – there is no NVIC involvement
    - The cause of interrupt from accelerometer is identified by reading appropriate registers through the I2C interface

ISR IMPLEMENTATION

iii. Do the BARE MINIMUM required to deal with the cause (continued in the next slide)

- If the interrupt was triggered because a new data is available - grab the data and store it in a buffer
- You might also want to set a boolean flag to indicate that a new data is available. This flag can be polled in the main program, which then does further processing on the data and clears the flag
- As a general rule, the higher the priority/urgency of the interrupt, the more effort you should put in to make the ISR shorter (execution time)
  - Generally, an interrupt which has a lower inter-arrival time is more urgent, as the likelihood of missing it is higher if not processed before it arrives again. This also depends on the application requirements

ISR IMPLEMENTATION

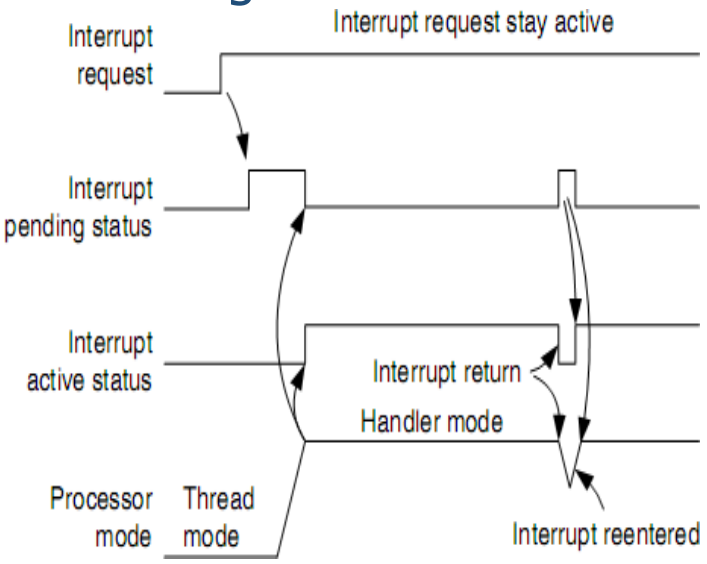
iii. Do the BARE MINIMUM required to deal with the cause

- Do not use any "blocking" code in the ISR. Blocking code refers to a code which 'waits' until certain condition is satisfied
  - e.g., waiting for a button press, HAL_Delay() by a certain amount of time
- Any variable/flag modified by the ISR should be qualified as volatile (for example, `volatile uint32_t uwTick;`) to ensure that the main function or its callees always get the latest data, i.e., reads and writes are not optimized away based on the assumption that it is not modified outside the main function

ISR IMPLEMENTATION

iv. Clear the interrupt (a.k.a clear pending status inside the interrupting device) (continued in the next slide)

- Clearing an interrupt is the process of telling/causing the interrupting device to de-assert the interrupt. The exact way in which it is done is dependent on the specific peripheral, and on the specific condition inside the peripheral which triggered the interrupt. This typically involves one of the following
  - reading a data register within the interrupting device (for data-ready type interrupts)
  - writing a data register within the device (for transmit register empty type interrupts)
  - reading a status register (for errors/status interrupts)
  - writing to an interrupt clear register (for errors/status interrupts)
  - Some interrupts arrive as pulses (e.g., SysTick); doesn't require any explicit clearing



ISR IMPLEMENTATION

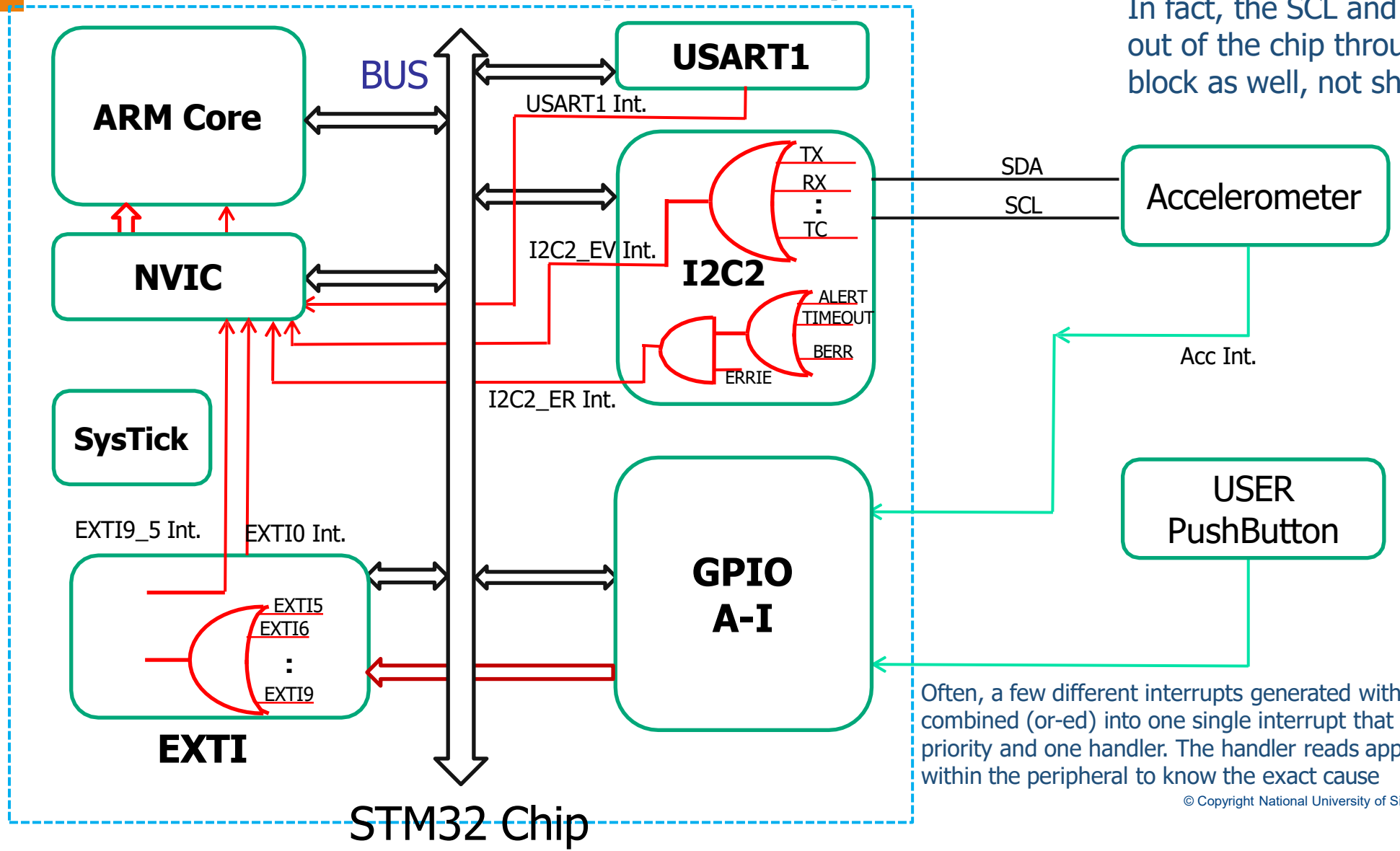
iv. Clear the interrupt (a.k.a clear pending status inside the interrupting device)

- If the interrupt is from an external device (e.g., a sensor) which is routed to NVIC through an EXTI, both EXTI interrupt and the external device interrupt should be cleared
- The clearing process is done through the normal data channel connecting the external device to the processor – there is no NVIC involvement
  - The interrupt from accelerometer is cleared by reading/writing appropriate registers within the accelerometer through I2C interface
  - Clearing the EXTI interrupt is done by writing to EXTI_PR through the internal bus
- Clearing pending status in NVIC ≠ disabling an interrupt in NVIC ≠ clearing the interrupt (i.e., clearing interrupt pending status inside the interrupting device)

ISR IMPLEMENTATION

v. POP all the registers pushed at the beginning of the handler before exiting

NVIC Connections (Partial)



In fact, the SCL and SDA comes out of the chip through GPIO block as well, not shown below

Often, a few different interrupts generated within a peripheral are combined (or-ed) into one single interrupt that goes into NVIC ->only one priority and one handler. The handler reads appropriate status register(s) within the peripheral to know the exact cause

INTERRUPTING DEVICE AND/OR PERIPHERAL CONFIGURATION

- The interrupting device/peripheral should be configured to generate an interrupt for the specific condition of interest
  - The interrupt should be enabled inside the interrupting device - the exact way in which this is done is device dependent
  - If the interrupt is from an external device (e.g. a sensor) which is routed to NVIC through an EXTI, we will need to enable both EXTI interrupts and the device interrupts
    - EXTI interrupts are enabled by writing to EXTI_IMR, EXTI_RTSR / EXTI_FTSR etc.
  - External device (e.g. sensor) interrupt is enabled by writing to the
  - appropriate register through the normal data channel connecting the device to the processor (e.g. I2C)

Not to be confused with the interrupts generated by the I²C interface (I2C_EV and I2C_ER)

Clear Pending and Enable Interrupt in NVIC

- Clear the pending status in NVIC
  - This step is optional, but desirable in many circumstances
- Enable the interrupt at NVIC
  - The interrupt for the device/peripheral should be enabled in NVIC so that NVIC can deal with the interrupt when it occurs
  - Else, the interrupt can go pending, but will not become active

Interrupt Handler Invocation

- Once everything is set up, interrupt triggering itself typically does not require any action from software
- The interrupt service routine is invoked automatically by the hardware provided appropriate conditions are met
  - It is not 'called' explicitly from the main program
  - However, there are circumstances where the software triggers an interrupt explicitly (e.g. for testing). An external interrupt can be triggered artificially by
    - Writing to ISPR
    - via Software Trigger Interrupt Register (STIR) in NVIC

Exceptions such as SVC (supervisor call) are triggered by software. SVC is used to transfer control to the Operating System (OS)

EXAMPLE

- PriorityGroup=5
- PreemptPriority=0b11
- SubPriority=0b00
- Full IPR contents: 0b11000000 = 0xC0

Only the NVIC side (dealing with interrupts) configuration is shown. EXTI configuration (generation of interrupts) was covered in previous sessions

Table 7.5 Definition of Preempt Priority Field and Subpriority Field in a Priority Level Register in Different Priority Group Settings

Priority Group	Preempt Priority Field	Subpriority Field
0	Bit [7:1]	Bit [0]
1	Bit [7:2]	Bit [1:0]
2	Bit [7:3]	Bit [2:0]
3	Bit [7:4]	Bit [3:0]
4	Bit [7:5]	Bit [4:0]
5	Bit [7:6]	Bit [5:0]
6	Bit [7]	Bit [6:0]
7	None	Bit [7:0]

SetPriority() expects right-aligned priority but IPR is left-aligned

```
NVIC_SetPriorityGrouping(5);
NVIC_SetPriority(EXTI15_10_IRQn, 0x0C);
// Set priority level to 0xC0 = 1100 0000
NVIC_ClearPendingIRQ(EXTI15_10_IRQn);
NVIC_EnableIRQ(EXTI15_10_IRQn);
```

```
#include "stm32l4s5xx.h " //contains definition of __NVIC_PRIO_BITS, xxxIRQns etc.
int main(void) {
        uint32_t ans, PG=5, PP=0b11, SP=0b00;
        NVIC_SetPriorityGrouping(5);
        ans = NVIC_EncodePriority(PG,PP,SP); // ans = 0x0C
        NVIC_SetPriority(EXTI15_10_IRQn,ans);
/* Priority in the IPR for IRQn = 40 set to 0xC0 (SetPriority shifts it up for us)
NVIC->IP[((uint32_t)IRQn)] = (uint8_t)((priority << (8U - __NVIC_PRIO_BITS)) & (uint32_t)0xFFUL);
*/
}
```

EXAMPLE

stm32l4xx_it.c

```
void EXTI15_10_IRQHandler(void)
{
/* For vector table to be set up correctly,
the interrupt handler name should match the
interrupt handler name in
startup_stm32l4s5xx.s*/
  HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_10);
  HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_11);
  HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_12);
  HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13);
  HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_14);
  HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_15);
}

void HAL_GPIO_EXTI_IRQHandler(uint16_t
GPIO_Pin)
{
  /* EXTI line interrupt detected */
  if(__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) !=
                                    0x00u)
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Callback(GPIO_Pin);
  }
}
```

```
__weak void HAL_GPIO_EXTI_Callback(uint16_t
                                   GPIO_Pin)
{
/* Prevent unused argument(s) compilation
warning */
  (void)(GPIO_Pin);

  /* NOTE: This function should not be
modified, when the callback is needed, the
HAL_GPIO_EXTI_Callback could be implemented
in the user file (main.c) */
}
```

main.c

```
// Implemented by the user in main.c
// this one overrides the _weak function
with the same name
void HAL_GPIO_EXTI_Callback(uint16_t
GPIO_Pin)
{
  // Actual handler implementation here
  // following the guidelines in slide 5-9
}
```

CONCLUSION

- Interrupt Programming steps
  - PRIGROUP and IPR setting
  - ISR implementation
  - Device/peripheral interrupt configuration
  - Clear past interrupt events in NVIC
  - Enabling interrupt in NVIC
- Interrupt Handler invocation
  - Automatically from hardware
  - Manually from software
- Example code